🔐Authentication & RBAC – Runbook and Technical Documentation

This document describes the **Authentication (Auth)** and **Role-Based Access Control (RBAC)** implementation for the Hotel SaaS backend.

It is intended for: - Backend developers - Future maintainers - DevOps / security review - Architectural reference

This document reflects the **current, completed MVP state**.

✂️Scope & Status

Scope Covered

- User authentication (login)
- JWT-based authorization
- Role-based access control (ADMIN / STAFF)
- Tenant-aware authentication

Status

✅**Auth: COMPLETE (MVP)**
✅**RBAC: COMPLETE (MVP)**

No changes are required before building domain modules.

✒️Authentication Overview

Authentication Model

- **Stateless authentication** using JWT
- **Email + password** login
- Passwords stored **hashed only**

Identity Concepts

Concept	Meaning
User	Hotel staff member
Tenant	Hotel
Role	Access level (ADMIN / STAFF)

🛌Login Flow (End-to-End)

Endpoint

```
POST /auth/login
```

Flow

1. User submits `email` + `password`
2. User is fetched by email
3. Password is validated using bcrypt
4. JWT access token is issued
5. Client uses JWT for subsequent requests

🌂JWT Token Design

JWT Payload

```
{
  "sub": "<userId>",
  "email": "user@example.com",
  "role": "ADMIN | STAFF",
  "tenantCode": "HOTEL_ABC"
}
```

Design Decisions

- `sub` used for user identification (JWT standard)
- `role` enables RBAC without DB lookup
- `tenantCode` enables tenant-aware logic

🌡️ JWT Strategy

Implementation

- Uses `passport-jwt`
- Token extracted from `Authorization: Bearer <token>` header
- Secret loaded via `ConfigService.getOrThrow`

Responsibilities

- Validate token signature
- Validate token expiration
- Attach decoded payload to `request.user`

🎒JwtAuthGuard

Purpose

- Enforces authentication on protected routes
- Blocks requests without valid JWT

Behavior

Condition	Result
Valid JWT	Request allowed
Missing JWT	401 Unauthorized
Invalid JWT	401 Unauthorized

Design

- Thin wrapper over `AuthGuard('jwt')`
- No business logic inside the guard

🎓Role-Based Access Control (RBAC)

Roles

Defined in `UserRole` enum: - `ADMIN` - `STAFF`

Roles are **single-role per user** in MVP.

🎩@Roles Decorator

Purpose

- Declares **which roles are allowed** to access a route
- Stores role metadata only (no logic)

Example

```
@Roles(UserRole.ADMIN)
```

This keeps controllers explicit and readable.

🎽 RolesGuard

Purpose

- Enforces role authorization
- Reads role metadata from `@Roles`
- Compares against `request.user.role`

Execution Order

1. `JwtAuthGuard` – authenticate user
2. `RolesGuard` – authorize user

Behavior

Role	Allowed	Result
ADMIN	Yes	200
STAFF	No	403 Forbidden
No JWT	No	401 Unauthorized

🕐 Guard Usage Pattern

Controllers apply guards at class level:

```
@UseGuards(JwtAuthGuard, RolesGuard)
```

Roles are applied at method level:

```
@Roles(UserRole.ADMIN)
```

This ensures: - Authentication is always enforced - Authorization is explicit per endpoint

---

✂️ Security Guarantees

Guaranteed by Design

- Passwords are never returned in API responses
- Passwords are always bcrypt-hashed
- JWT secrets are not hardcoded
- Unauthorized access is blocked early
- Role escalation is not possible via API

✂️✒️ Known Limitations (Intentional)

The following are **intentionally deferred**:

Feature	Reason
Refresh tokens	Not required for MVP
Logout / revocation	Needs persistence layer
Rate limiting	Infrastructure concern
Permission-based RBAC	Overengineering for MVP
Audit logs	Add after domain modules

✂️🛏️ Operational Runbook

Environment Variables Required

```
JWT_SECRET=
JWT_EXPIRES_IN=
```

Running Auth Locally

```
npm run start:dev
```

Common Issues

JWT Always Invalid

- Check `JWT_SECRET`
- Check token expiration
- Ensure `JwtStrategy` is registered

Role Guard Blocking Access

- Verify `@Roles()` decorator
- Verify user role in database
- Ensure `JwtAuthGuard` runs before `RolesGuard`

✂️☂️ Architectural Verdict

- Auth implementation is **production-grade for MVP**
- RBAC is **correctly layered and extensible**
- Safe to build all business modules on top
- No refactor required

🔗Final Status

🚷**Authentication: COMPLETE**
🚻**RBAC: COMPLETE**
🚷**Ready for domain modules (Rooms, Reservations, Billing)**

End of Auth & RBAC Documentation